PROJECT BRIEF — API PLATFORM

RegRadar

Regulatory & Compliance Change Monitoring API

Full Feature Specification · Tech Stack · Hosting Architecture · May 2026

What Is RegRadar?

RegRadar is a developer-first API that monitors regulatory and compliance changes across jurisdictions and industries in real time. Developers subscribe to a combination of **jurisdiction + industry** (e.g. India + Fintech, EU + Healthcare), and whenever a law, regulation, guideline, or compliance requirement changes, their application receives a structured webhook payload with a plain-language summary, a machine-readable diff, source links, and an effective date — all without writing a single scraper.

1. Why This Exists — The Problem

Compliance teams at fintechs, hospitals, law firms, and insurance companies manually track regulatory changes across dozens of government portals, PDFs, and email newsletters. Missing a single change can cost millions in fines or forced product shutdowns.

The Pain	The Current 'Solution'	The Cost of Getting It Wrong
Laws change silently on government portals	Junior staff manually checks 40+ sites weekly	Fines of $50K–$10M+ per violation
Regulations differ by jurisdiction	Spreadsheets, Slack threads, email alerts	Regulatory shutdowns, license revocations
PDFs and legalese are hard to parse	$50K+/yr enterprise tools (Thomson Reuters)	Lawsuits, reputational damage

No structured diff between old and new rules	In-house legal teams reading dense documents	Delayed product launches

2. Full Feature Set

2.1 Core API Features

Subscription Management	Developers register subscriptions via POST /subscriptions with a jurisdiction (ISO 3166 country/region codes), industry vertical, and optional sub-topic filters. Multiple subscriptions per API key. Pause, resume, or delete subscriptions at any time.
Real-Time Webhook Delivery	When a change is detected, RegRadar fires a signed webhook to the developer's registered endpoint within minutes. Payload includes: plain-language summary, structured diff (added/removed/modified clauses), source URL, effective date, severity level (critical / major / minor), and a unique change ID.
Change Feed (Poll)	For developers who prefer polling over webhooks, GET /changes returns a paginated, filterable feed of all detected changes. Supports cursor-based pagination for reliability.
Source Verification	Every change is traced back to an official government, regulatory body, or authoritative legal source. No blogs, no secondhand summaries. Sources are linked and archived with timestamps.
Severity Scoring	Each change is automatically scored: Critical (immediate action required), Major (action within 30 days), or Minor (informational). Score is generated by AI analysis of the change's scope and penalty clauses.
Plain-Language Summaries	Dense legalese is translated into a plain-language summary (3–5 sentences) suitable for displaying directly to end-users or compliance officers in your product.
Structured Diff Output	Machine-readable before/after diff of the regulation. Returned as JSON with added[], removed[], and modified[] arrays — each with the original text, new text, and clause reference.
Search & Query	GET /search allows full-text search across the entire regulatory change history. Filter by jurisdiction, industry, date range, severity, or keyword.
Historical Archive	Full history of every regulatory change RegRadar has ever detected, accessible via API. Useful for audits, compliance reports, and building timelines.

2.2 Developer Experience Features

Feature	Description
Interactive Playground	Web UI to test queries, preview webhook payloads, and explore jurisdictions without writing code.
Python + JS/TS SDKs	Official SDKs on PyPI and npm. One-line install, typed responses, built-in retry logic and webhook signature verification.
OpenAPI / Swagger Docs	Auto-generated, interactive API reference. Hosted at docs.regradar.dev.
Webhook Simulator	Trigger test webhook payloads to any URL directly from the dashboard — no need to wait for a real regulatory change to test your integration.

Usage Dashboard	Real-time view of API calls, webhook delivery status, subscription health, latency metrics, and rate limit usage.
Alert Digest Emails	Optional daily or weekly email digests for non-developer stakeholders who need updates but don't consume the API directly.

3. Sample API Payloads

3.1 Create a Subscription — Request

```
POST https://api.regradar.dev/v1/subscriptions

Authorization: Bearer YOUR_API_KEY

Content-Type: application/json

{

"jurisdiction": "IN", // ISO 3166 country code (India)

"industry": "fintech",

"topics": ["payment_systems", "KYC", "AML"],

"webhook_url": "https://yourapp.com/webhooks/compliance",

"severity_min": "major" // only fire for major or critical changes

}
```

3.2 Webhook Payload — Change Detected

```
{

"event": "regulation.changed",

"change_id": "chg_01HWXYZ9K2B3N",

"jurisdiction": "IN",

"industry": "fintech",

"topic": "KYC",

"severity": "critical",

"source": {

"authority": "Reserve Bank of India",

"url": "https://rbi.org.in/circulars/2026/kyc-update.pdf",

"archived_at": "2026-05-10T09:14:00Z"

},

"summary": "RBI now requires video-KYC re-verification for all accounts",

"dormant for more than 12 months. Effective 1 July 2026.",

"effective_date":"2026-07-01",

"diff": {

"added": [{"clause": "Section 4.2b", "text": "Video KYC re-verification..."}],

"removed": [],

"modified":[{"clause": "Section 4.1", "old": "...", "new": "..."}]

},

"signature": "sha256=abc123..."

}
```

4. Tech Stack

4.1 Backend

Component	Technology	Why
Language	Python 3.12	Mature async ecosystem; great for scraping + AI pipelines
API Framework	FastAPI	Auto OpenAPI generation, async-native, typed responses
Task Queue	Celery + Redis	Background jobs for scraping, parsing, webhook delivery
Message Broker	Redis (also cache)	Fast pub/sub for webhook fan-out; session + rate-limit cache
Database	PostgreSQL 16	Reliable, structured storage for subscriptions + change history
Search Index	Elasticsearch 8	Full-text search across regulatory change history
AI / NLP Layer	Claude API (Anthropic)	Plain-language summaries, severity scoring, diff analysis
Scraping Engine	Playwright + httpx	Headless browser for JS-rendered gov sites; httpx for REST
Webhook Delivery	Python + HMAC-SHA256	Signed payloads, retry logic with exponential backoff
Auth	API Key + JWT	API keys for machine access; JWT for dashboard sessions

4.2 Frontend (Developer Dashboard)

Component	Technology	Why
Framework	Next.js 15 (React)	Fast, SSR-capable, easy deployment on Vercel
Styling	Tailwind CSS	Rapid UI development, consistent design system
API Client	TanStack Query	Caching, polling, and async state management
Charts / Metrics	Recharts	Lightweight, composable charts for usage dashboard
Auth UI	NextAuth.js	OAuth + email login for dashboard access
Code Highlighting	Shiki	Syntax-highlighted API response previews in docs
Docs Site	Fumadocs	OpenAPI-integrated docs with interactive try-it console

4.3 Infrastructure & DevOps

Layer	Technology	Why
Containerisation	Docker + Docker Compose	Consistent dev/prod parity; easy local setup
Orchestration	Kubernetes (K8s)	Scale scraper workers and API independently
CI/CD	GitHub Actions	Automated test, build, and deploy pipeline
Secrets Management	HashiCorp Vault	Secure API key and credential storage
Monitoring	Prometheus + Grafana	Real-time metrics: latency, error rates, queue depth

Logging	Loki + Grafana	Centralised log aggregation and alerting
Error Tracking	Sentry	Exception capture with full stack traces

5. Hosting Architecture

5.1 Cloud Provider Recommendation

The recommended hosting setup uses **AWS** as the primary cloud provider, with **Vercel** for the frontend dashboard and docs site. This combination gives maximum reliability, global edge delivery for the dashboard, and fine-grained control over the backend infrastructure.

Service / Component	AWS Service Used	Purpose
API Server (FastAPI)	ECS Fargate (containerised)	Serverless containers — scale to zero when idle
Celery Workers (scrapers)	ECS Fargate (spot tasks)	Cost-optimised background scraping jobs
PostgreSQL Database	RDS Aurora Serverless v2	Auto-scaling, managed Postgres
Redis Cache / Broker	ElastiCache (Redis OSS)	Task queue + rate limiting + session cache
Elasticsearch	Amazon OpenSearch Service	Managed search cluster
File / Archive Storage	S3 + Glacier	Archived source documents (PDFs, HTML snapshots)
CDN	CloudFront	API edge caching; low-latency global delivery
DNS	Route 53	Domain management + health-check routing
Secrets	AWS Secrets Manager	API keys, DB credentials, webhook signing secrets
Frontend (Dashboard)	Vercel	Zero-config Next.js hosting, global edge network
Docs Site	Vercel	Same deployment as dashboard

5.2 Architecture Diagram (Logical Flow)

```
Developer App --> POST /subscriptions --> RegRadar API (FastAPI / ECS Fargate)
|
PostgreSQL (RDS Aurora)

Scraper Workers (ECS Fargate Spot) --> Playwright / httpx
| fetch official gov/regulatory sources every 15–60 min
|
v
Change Detected --> Claude API (summarise + severity score + diff)
|
v
Change stored in PostgreSQL + indexed in OpenSearch
|
+--> Celery publishes to Redis queue
|
v
```

```
Webhook Worker --> POST to Developer's webhook_url (signed HMAC-SHA256)
|
v
Retry queue (exponential backoff: 1m, 5m, 30m, 2h, 24h)

Developer Dashboard (Next.js / Vercel) <--> RegRadar API <--> PostgreSQL / OpenSearch
```

5.3 Environments

Environment	Purpose	Infra	Cost Est./Month
Local Dev	Developer laptops	Docker Compose (all services local)	$0
Staging	Testing before release	ECS Fargate (minimal), RDS t3.micro	~$40–80
Production	Live API	ECS Fargate auto-scale, Aurora Serverless, OpenSearch	~$150–400

6. Monetisation & Pricing Tiers

Tier	Price	Rate Limit	Subscriptions	Features
Free	$0/month	500 API calls/month	2	Webhooks, 7-day history, community support
Starter	$49/month	10,000 calls/month	10	+ 1yr history, email digests, SDK access
Pro	$199/month	100,000 calls/month	Unlimited	+ full history, priority webhooks, SLA 99.9%
Enterprise	Custom	Unlimited	Unlimited	+ private sources, custom jurisdictions, SLAs

7. Build Roadmap

Phase 1 — MVP (Weeks 1–6)

- Core FastAPI backend with /subscriptions, /changes, /search endpoints
- PostgreSQL schema: subscriptions, changes, sources, webhook_deliveries
- Scraper engine for 5–10 seed sources (RBI, SEBI, MCA for India fintech)
- Claude API integration for summary + severity scoring
- Basic webhook delivery with HMAC signing and retry logic
- Docker Compose local dev setup
- Minimal dashboard: API key management, subscription list

Phase 2 — Developer Ready (Weeks 7–10)

- Python SDK (PyPI) and TypeScript SDK (npm)
- Interactive playground + webhook simulator
- OpenAPI docs site (Fumadocs)
- Elasticsearch integration for full-text search
- Staging environment on AWS (ECS + RDS)
- Rate limiting middleware (Redis-backed)
- Usage dashboard with charts

Phase 3 — Production & Growth (Weeks 11–16)

- Production AWS deployment (ECS Fargate + Aurora Serverless)
- Stripe billing integration for paid tiers
- Expand to 50+ regulatory sources across 5 jurisdictions
- Prometheus + Grafana monitoring stack
- Sentry error tracking
- Launch on Product Hunt + reach out to 20 fintech startups
- SLA dashboard for Pro/Enterprise customers

8. Competitive Advantage Summary

Why RegRadar wins against existing options:

Factor	Thomson Reuters / Lexis	Manual Monitoring	RegRadar
Price	$50K+/yr	Staff cost: $80K+/yr	$49–199/month
Developer API	None / complex	None	First-class REST API
Structured JSON output	No	No	Yes — typed schemas
Webhook delivery	No	No	Yes — real-time
Setup time	Months	Weeks	< 30 minutes
SMB / Startup friendly	No	No	Yes
Coverage expansion	Fixed	Manual	Continuous + API-driven

Bottom Line: RegRadar is a focused, buildable, solo-developer-friendly project with a clear monetisation path, zero direct competitors at the API layer, and customers who have demonstrated willingness to pay for compliance tooling. The tech stack is modern, well-documented, and deployable at low cost. This document is your build bible — start with Phase 1, get one fintech startup using it for free, and iterate from there.